LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – Relasi Antar Kelas

GEUGEUT NYARIKAWANTI SURAHMAT

103132400002

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

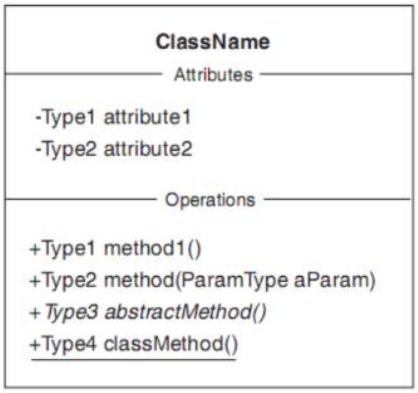
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Diagram Kelas

Diagram kelas merupakan sebuah diagram yang digunakan untuk memodelkan kelas-kelas yang digunakan di dalam sistem beserta hubungan antar kelas dalam sistem tersebut. Kelas dalam diagram kelas digambarkan sebagai berikut:



2. Hubungan Antar Kelas

Dalam Object Oriented Programming, kelas-kelas yang terbentuk dapat memiliki hubungan satu dengan yang lainnya, sesuai dengan kondisi dari kelas-kelas yang bersangkutan. Ada beberapa jenis hubungan yang dapat terjadi antara kelas yang satu dengan kelas yang lainnya, antara lain:

a. Asosiasi

Asosiasi merupakan hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas. Jika dua kelas dalam suatu model perlu berkomunikasi satu sama lain, harus ada hubungan di antara kelas hal tersebut dapat diwakili oleh asosiasi.

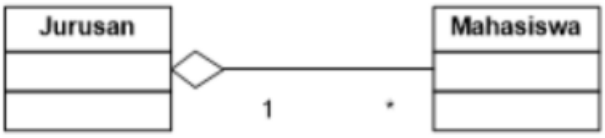
Hubungan asosiasi disimbolkan sebagai berikut.



Pada diagram kelas di atas, terlihat hubungan antara kelas Dosen dengan kelas Mahasiswa. Method pada Kelas Mahasiswa dipanggil pada kelas Dosen sehingga memiliki hubungan asosiasi.

b. Agregasi

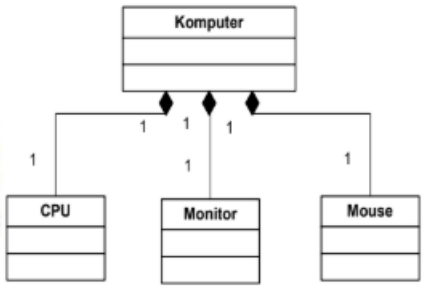
Agregasi merupakan hubungan antara dua kelas yang lebih ketat dan terdapat hubungan bagian (part) dan keseluruhan (whole). Contoh penerapan hubungan agregasi adalah penggunaan atribut bertipe kelas tertentu pada kelas lain, disimbolkan dengan:



Contoh penerapan relasi agregasi antara kelas jurusan dan mahasiswa adalah penggunaan atribut bertipe kelas Mahasiswa pada kelas Jurusan.

c. Komposisi

Komposisi merupakan bentuk khusus dari agregasi dimana terdapat hubungan antar kelas yang lebih kuat, karena yang menjadi part (bagian) tidak akan ada tanpa adanya kelas yang menjadi whole.



Dari diagram kelas di atas terlihat bahwa kelas CPU, Monitor, dan Mouse semuanya merupakan bagian dari kelas Komputer dan ketika kelas Komputer musnah maka kelas CPU, Monitor, dan Mouse akan ikut musnah.

B. Guided

1. Agregasi

a. Jurusan.java

```
package com.geugeut.guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, NamaJurusan;
    private Mahasiswa[] daftarMahasiswa= new Mahasiswa[32];

    //constructor
    public Jurusan(String kodeJurusan, String NamaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.NamaJurusan = NamaJurusan;
    }
    //behaviour
    public void tambahMahasiswa(Mahasiswa m){
        daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }
    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.NamaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i < jumlahMahasiswa; i++){
            System.out.println(i+1 + ". " +
daftarMahasiswa[i].getNIM() + " - " +
daftarMahasiswa[i].getNama());
```

```
        }
    }
    //getter
    public String getKodeJurusan() {
        return this.kodeJurusan;
    }
    public String getNamaJurusan() {
        return this.NamaJurusan;
    }
    //setter
    public void setKodeJurusan(String kodeJurusan) {
        this.kodeJurusan = kodeJurusan;
    }
    public void setNamaJurusan(String NamaJurusan) {
        this.NamaJurusan = NamaJurusan;
    }
    // Method main untuk menjalankan dan menguji relasi agregasi
    public static void main(String[] args) {
        // 1. Buat objek Jurusan
        Jurusan informatika = new Jurusan("IF01", "Informatika");

        // 2. Buat objek Mahasiswa (konsep agregasi: mahasiswa
dibuat mandiri di luar jurusan)
        Mahasiswa mhs1 = new Mahasiswa("22001", "Caca");
        Mahasiswa mhs2 = new Mahasiswa("22002", "Andi");

        // 3. Masukkan mahasiswa ke jurusan
        informatika.tambahMahasiswa(mhs1);
        informatika.tambahMahasiswa(mhs2);

        // 4. Tampilkan daftar mahasiswa dalam jurusan
```

```
        informatika.displayMahasiswa();
    }
}
```

b. Mahasiswa.java

```
package com.geugeut.guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama) { //ini constructor
        this.NIM = NIM;
        this.Nama = Nama;
    }
    //getter
    public String getNIM() {
        return this.NIM;
    }
    public String getNama() {
        return this.Nama;
    }
    //setter
    public void setNIM(String NIM) {
        this.NIM = NIM;
    }
    public void setNama(String Nama) {
        this.Nama = Nama;
    }
}
```

2. Asosiasi

a. Dosen.java

```
package com.geugeut.guided.asosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    // Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa2 s, int nilai) {
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa2 s) {
```

```
        return s.getNilai();
    }
}
```

b. Mahasiswa2.java

```
package com.geugeut.guided.asosiasi;

public class Mahasiswa2 {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }
    public int getNilai() {
        return this.nilai;
    }
    public String getNama() {
        return this.nama;
    }

    public String getNim(){
        return this.nim;
    }
```

```
}
```

3. Komposisi

a. Komputer.java

```
package com.geugeut.guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String
brandMonitor, String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

b. Monitor.java

```
package com.geugeut.guided.komposisi;

public class Monitor {
    private String brand;
```

```
    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

c. Mouse.java

```
package com.geugeut.guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

d. Processor.java

```
package com.geugeut.guided.komposisi;

public class Processor {
    private String brand;
```

```
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Output Guided:

```
Nim :130118383
Nama :Budi
Nilai :90
```

C. Unguided

1. Soal Unguided 1

Buatlah program Java untuk sistem manajemen pesanan cafe dengan menerapkan tiga jenis relasi antarclass, yaitu komposisi (composition), agregasi (aggregation), dan asosiasi (association). Pada class RincianHarga, buat instance field hargaDasar dan pajak bertipe double, serta method getHargaTotal() untuk menghitung total harga setelah pajak 10%. Pada class ItemPesanan, buat instance field namaItem bertipe String dan rincian bertipe RincianHarga. Objek RincianHarga harus diinstansiasi langsung di dalam constructor ItemPesanan untuk menerapkan komposisi. Pada class Pesanan, buat instance field nomorMeja bertipe int, daftarItem berupa array ItemPesanan, dan jumlahItem sebagai penanda jumlah item yang tersimpan. Class Pesanan memiliki method tambahItem() untuk memasukkan objek ItemPesanan ke dalam array serta method tampilkanPesanan() untuk menampilkan daftar item beserta

harga totalnya. Pada class Kasir, buat instance field namaKasir bertipe String dan method prosesPembayaran() yang menerima objek Pesanan sebagai parameter, menghitung total subtotal dan pajak seluruh item, serta mencetak struk pembayaran tanpa menyimpan objek Pesanan sebagai field. Di class Main, buat satu objek Kasir bernama Budi, tiga objek ItemPesanan untuk Iced Americano (harga dasar Rp18.000), Beef Croissant (Rp25.000), dan Matcha Latte (Rp22.000), serta satu objek Pesanan untuk Meja 4 dengan kapasitas array 5 item. Masukkan ketiga item ke dalam pesanan menggunakan method tambahItem(), tampilkan rincian pesanan, lalu jalankan proses pembayaran dan tampilkan total subtotal, total pajak, serta total bayar. Gunakan enkapsulasi, constructor dengan penggunaan this secara eksplisit, dan array standar bertipe tetap tanpa menggunakan ArrayList. Tidak perlu menggunakan inheritance, overriding, maupun abstract class.

Jawaban :

Kasir.java

```
package com.geugeut.unguided;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftarItem = pesanan.getDaftarItem();
        int jumlahItem = pesanan.getJumlahItem();
```

```
            for (int i = 0; i < jumlahItem; i++) {
                RincianHarga rincian = daftarItem[i].getRincian();

                totalSubtotal += rincian.getHargaDasar();
                totalPajak += rincian.getPajak();
            }

            double totalBayar = totalSubtotal + totalPajak;

            System.out.println("=== STRUK PEMBAYARAN KASIR ===");
            System.out.println("Kasir Bertugas : " + namaKasir);
            System.out.println("Pesanan        : Meja " +
        pesanan.getNomorMeja());
            System.out.println("Jumlah Item    : " + jumlahItem);
            System.out.println("Total Subtotal : Rp " + totalSubtotal);
            System.out.println("Total Pajak    : Rp " + totalPajak);
            System.out.println("TOTAL BAYAR    : Rp " + totalBayar);

        System.out.println("================================");
            }
}
```

ItemPesanan.java

```
        package com.geugeut.unguided;

        public class ItemPesanan {
            private String namaItem;
            private RincianHarga rincian;

            public ItemPesanan(String namaItem, double hargaDasar) {
                this.namaItem = namaItem;
```

```
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
    }
}
```

Pesanan.java

```
package com.geugeut.unguided;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
            jumlahItem++;
        } else {
            System.out.println("Pesanan sudah penuh.");
```

```
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + nomorMeja);

        for (int i = 0; i < jumlahItem; i++) {
            ItemPesanan item = daftarItem[i];
            RincianHarga rincian = item.getRincian();

            System.out.println(
                (i + 1) + ". " + item.getNamaItem()
                + " - Rp " + rincian.getHargaTotal()
                + " (Harga: " + rincian.getHargaDasar()
                + ", Pajak: " + rincian.getPajak() + ")"
            );
        }

        System.out.println();
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

RicianHarga.java

```
package com.geugeut.unguided;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public double getPajak() {
        return pajak;
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

Main.java

```
package com.geugeut.unguided;

public class Main {
    public static void main(String[] args) {
        // Membuat objek kasir
        Kasir kasir = new Kasir("Budi");
```

```
        // Membuat item pesanan secara independen
        ItemPesanan item1 = new ItemPesanan(
            "Iced Americano", 18000
        );

        ItemPesanan item2 = new ItemPesanan(
            "Beef Croissant", 25000
        );

        ItemPesanan item3 = new ItemPesanan(
            "Matcha Latte", 22000
        );

        // Membuat pesanan untuk meja 4
        Pesanan pesanan = new Pesanan(4, 5);

        // Memasukkan item ke dalam pesanan
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // Menampilkan daftar pesanan
        pesanan.tampilkanPesanan();

        // Memproses pembayaran
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Penjelasan :

Program Sistem Manajemen Pesanan Cafe digunakan untuk mencatat pesanan pelanggan, menghitung harga menu beserta pajak 10%, dan memproses pembayaran. Program terdiri atas lima class, yaitu RincianHarga, ItemPesanan, Pesanan, Kasir, dan Main, untuk mengimplementasikan konsep komposisi (composition), agregasi (aggregation), dan asosiasi (association).

Secara umum, alur program adalah sebagai berikut:

1. Menyimpan rincian harga pada class RincianHarga. Class ini memiliki atribut hargaDasar dan pajak bertipe double. Pajak dihitung sebesar 10% dari harga dasar, sedangkan method getHargaTotal() mengembalikan hasil penjumlahan harga dasar dan pajak.
2. Membuat item pesanan pada class ItemPesanan. Class ini menyimpan nama menu dan objek RincianHarga. Objek RincianHarga dibuat langsung di dalam constructor ItemPesanan, sehingga menerapkan konsep komposisi, yaitu objek rincian harga menjadi bagian internal dari item pesanan.
3. Mengelola pesanan pada class Pesanan. Class ini menyimpan nomor meja, array daftarItem, dan variabel jumlahItem. Method tambahItem() memasukkan objek ItemPesanan yang telah dibuat sebelumnya ke dalam array, sedangkan method tampilkanPesanan() menampilkan daftar menu beserta rincian

harganya. Proses ini menerapkan konsep agregasi karena objek item dibuat secara terpisah sebelum dimasukkan ke dalam pesanan.

4. Memproses pembayaran pada class Kasir. Method prosesPembayaran() menerima objek Pesanan sebagai parameter, kemudian menghitung total subtotal, total pajak, dan total pembayaran seluruh item. Class Kasir tidak menyimpan objek Pesanan sebagai atribut, sehingga interaksi keduanya menerapkan konsep asosiasi.
5. Menginisialisasi dan menjalankan program pada class Main. Pada method main(), dibuat satu objek Kasir, tiga objek ItemPesanan untuk Iced Americano, Beef Croissant, dan Matcha Latte, serta satu objek Pesanan untuk Meja 4. Ketiga item dimasukkan ke dalam pesanan menggunakan method tambahItem(). Program kemudian menampilkan daftar pesanan dan mencetak struk pembayaran yang berisi subtotal Rp65.000, total pajak Rp6.500, dan total bayar Rp71.500.

Program menerapkan enkapsulasi dengan menggunakan akses private pada atribut serta menyediakan getter untuk mengakses data yang diperlukan. Constructor menggunakan kata kunci this secara eksplisit untuk membedakan atribut objek dengan parameter yang memiliki nama sama. Selain itu, program menggunakan array standar bertipe tetap tanpa ArrayList, serta tidak menerapkan inheritance, overriding, maupun abstract class.

D. Kesimpulan

Praktikum ini memberikan pemahaman mengenai konsep Pemrograman Berorientasi Objek (PBO), khususnya penerapan hubungan antar kelas dalam bahasa Java yang meliputi komposisi (composition), agregasi (aggregation), dan asosiasi (association). Melalui pembuatan program Sistem Manajemen Pesanan Cafe, hubungan komposisi diterapkan pada kelas ItemPesanan dan RincianHarga, agregasi diterapkan pada kelas Pesanan yang menampung objek ItemPesanan menggunakan array, sedangkan asosiasi diterapkan pada kelas Kasir yang berinteraksi dengan objek Pesanan melalui parameter method. Selain itu, praktikum ini memperlihatkan penerapan enkapsulasi menggunakan access modifier private, penggunaan constructor dengan kata kunci this`, serta

pemanfaatan method untuk mengelola data dan memproses pembayaran. Dengan demikian, praktikum ini membantu memahami bagaimana hubungan antar kelas dapat dirancang dan diimplementasikan untuk membentuk program yang terstruktur sesuai dengan kebutuhan sistem.